Each object detection model is attached to a configuration which needs to be passed to the export_model.py. The export model stores facts such as the model architecture, graphical outputs of the object detection and establishes checkpoints so that the system can arrive straight to the node where it stopped learning rather than repeating its steps when it fails to converge on high accuracy.

The get_configs_from_pipeline_file method is used to create a dictionary from the configuration file, and the next keyword establishes a method to make a proto buffer object through the dictionary.

Coming to input_checkpoint, it specifies the path to model.ckpt of the trained model. model_version_id is simply an identification integer for the current version of the model that needs to be separately specified by the TensorFlow mainframe.

And finally, the object_detection.exporter will save the model as per the format specified in the code.

What will the execution produce

Congratulations, you have now reached the far end of the tutorial where the executions will yield the desired results as long as the code has been tested well and the datasets have been well trained.

Vehicle Detection In TensorFlow

The user will find the system outputting the results in the folder defined in the code with images that were inputted from the training set and the testing set.

You will see that the images have all been distinguished with a square or rectangular demarcation across all unique objects. Since we were working with cars for the entire length of this tutorial, the system will create navigational tags for the objects that closely resemble a car. It's worth noting that these images can again be trained to look for another kind of objects as long as it is mentioned in the code with the right training sets being fed to it.

You will also see that the images with the boxes have percentage values affixed to them which indicate their 'likeness' to the actual object that was